# Quantum-enhanced transmission-loss detection

Numerical implementation and comparison with the paper

Bhagyesh Kamal

30 October 2026

**Abstract**

I implemented the main measurement models and detection algorithms from Guha et al.'s paper on transmission-loss detection. My calculations reproduce the analytical curves in Figure 2 and the quoted single-photon relative entropy. The detection-delay simulations do not agree as closely: some published values remain unexplained. I describe the implementation, compare the results with the released data, and examine how false-alarm calibration and channel uncertainty affect the comparison. This is a partial numerical reproduction carried out on a classical computer.

## 1 Aim of the project

The question in Guha et al. [1] is whether a receiver can detect a small increase in optical loss sooner by adding squeezed light or entanglement to the transmitted pulses. The pulses can still carry a communication signal. In the favourable regime considered in the paper, the improvement approaches $1/(1-\eta_1)$, which is ten for $\eta_1 = 0.9$. Keeping the added energy per pulse fixed while increasing the entangled group size requires stronger seed squeezing.

I focused on the numerical part: generating receiver measurements, running CUSUM and comparing detection delays at the same false-alarm target. I checked my results against the equations and the authors' released data [2]. Some calculations agree closely; others still need clarification. The code and saved simulation data (release v0.1.0) are available at `https://github.com/Rigur-Calypso/quantum-loss-change-detection`.

## 2 Implementation

### 2.1 Coherent and squeezed pulses

Following the paper, I use $\eta_1$ and $\eta_2$ for transmission before and after the change, with $\eta_2 < \eta_1$. The first changed pulse is $\nu$. I compare probes with the same total energy: $N + N_a$ for the coherent baseline, and $N$ in the coherent signal plus $N_a$ in the quantum addition. The homodyne measurements follow

$$X \mid j \sim \mathcal{N}(\mu_j, v_j), \tag{1}$$

$$\mu_j^{\mathrm{coh}} = \sqrt{\eta_j}\sqrt{N + N_a}, \qquad v_j^{\mathrm{coh}} = \tfrac{1}{4}, \tag{2}$$

$$\mu_j^{\mathrm{sq}} = \sqrt{\eta_j}\sqrt{N}, \qquad v_j^{\mathrm{sq}} = \frac{\eta_j e^{-2r} + 1 - \eta_j}{4}, \qquad r = \operatorname{asinh}\sqrt{N_a}. \tag{3}$$

Here $v_j$ is the variance, and $j$ selects the before- or after-change law. I use natural logarithms, so KL is measured in nats.

The Gaussian likelihood and KL calculations are

$$\ell(x) = \log \frac{p_2(x)}{p_1(x)} = \frac{1}{2} \log \frac{v_1}{v_2} + \frac{(x-\mu_1)^2}{2v_1} - \frac{(x-\mu_2)^2}{2v_2}, \tag{4}$$

$$D_{\mathrm{KL}}(P_2 \| P_1) = \frac{1}{2} \left[ \frac{v_2}{v_1} - 1 - \log \frac{v_2}{v_1} + \frac{(\mu_2 - \mu_1)^2}{v_1} \right]. \tag{5}$$

The variance changes with loss in the squeezed case, so I include it in the likelihood rather than testing only for a shift in the mean.

### 2.2 Entangled pulse groups

For a group of $n$ pulses, the squeezed seed has energy $nN_a$, giving $s = \operatorname{asinh} \sqrt{nN_a}$. When all pulses pass through the same transmission, their joint Gaussian law has $u = (1, \ldots, 1)^{\mathsf{T}}/\sqrt{n}$ and

$$m_j = \sqrt{\eta_j N}\, \mathbf{1}, \qquad K_j = \tfrac{1}{4} I + (v_j - \tfrac{1}{4}) u u^{\mathsf{T}}, \qquad v_j = \frac{1 - \eta_j + \eta_j e^{-2s}}{4}. \tag{6}$$

For this homogeneous block, the likelihood depends on the collective measurement $u^{\mathsf{T}} X$. I divide the joint KL by $n$ to compare information per pulse, and check the result using the full covariance matrices.

For detection after each pulse, I use its distribution conditional on the earlier measurements in that block. Resetting the CUSUM score does not erase those correlations. If the loss changes halfway through a block, the simulation uses the corresponding mixed-loss covariance. I have not proved optimality for this correlated version of the detector.

### 2.3 CUSUM and the quantities I measure

I use Page's CUSUM recursion [3] for independent observations:

$$W_0 = 0, \qquad W_t = \max\{0, W_{t-1} + \ell(X_t)\}, \qquad T = \inf\{t : W_t > h\}. \tag{7}$$

The detector stops at the first strict crossing. Pulse indices start at one; if change-point estimates tie, I choose the earliest prefix.

I measure false alarms using the no-change average run length, and report delay conditional on reaching the true change without an earlier alarm:

$$\mathrm{ARL} = \mathbb{E}_\infty[T], \qquad d_\nu = \mathbb{E}_\nu[T - \nu \mid T \geq \nu]. \tag{8}$$

An alarm on pulse $\nu$ has zero delay. I record earlier alarms separately and do not restart those runs. When a run reaches the observation horizon without an alarm, I report a restricted mean. The conditional delay $d_\nu$ above is different from Lorden's worst-case criterion [4].

I calibrate thresholds using no-change simulations, then fix them before checking independent runs. I do not tune them to match the paper's delays. For very rare false alarms, I also use regenerative importance sampling. The reported uncertainty estimates and numerical checks are approximate.

## 3 Agreement with the paper

- **Figure 2.** I recalculated the coherent, squeezed and $n = 2, 4, \ldots, 256$ entangled KL curves for $N = 100$, $\eta_1 = 0.9$, $\eta_2 = 0.85$ and $N_a \in [0, 5]$. The closed forms and Gaussian calculations agree. This checks the formulas; the released dataset has no numeric Figure 2 array for a direct comparison.

- **Figure 11(a).** Integrating the single-photon measurement law gives 0.2063404792 nats, matching the paper's 0.2063. I also implemented its CDF and an exact sampler. The squeezing crossover is 1.812 dB, close to the paper's rounded 1.83 dB.
- **Delay scaling.** I tested fourteen coherent/squeezed settings with target ARLs 100–$10^8$. At high ARL, the delay slopes agree with inverse-KL scaling within 0.014%. There are still finite-start offsets; this numerical check does not prove the worst-case result.
- **Other checks.** The Hadamard covariance calculations agree below $2.8 \times 10^{-17}$ in the tested cases. At 1 dB, seven approximate values read from Figure 12 differ by at most 1.67%. I have not checked the full published curves point by point.

# 4 Results I could not match

Table 1 lists the comparisons I could not match. Relative differences use the authors' value as the denominator. For Figure 3, I average the absolute relative differences; for Figure 7, I average the signed differences within each family. I cannot conclude from these gaps alone that the published results are wrong.

Table 1: Differences between my results and arXiv v2.

| Comparison | My result | Remaining question |
|---|---|---|
| Figure 3 | The eight KL-ratio coordinates agree to numerical precision. Mean absolute relative delay-ratio differences are 7.49% without modulation and 6.40% with BPSK. | The KL values agree, but the simulated delay ratios still differ. |
| Figure 4 | At fixed seed squeezing, I obtain $S_n = A/n + B$ with $A \geq 0$, hence $S_1 \geq S_2 \geq S_4$. The published inset places $n = 2$ above $n = 1$. | I have not found the reason for this different ordering. |
| Figure 5 | My photon-number-resolving Kennedy KL values are 11.9–12.6 times four values read from the published plot. Independent integration and sampling agree. | Changing to on/off detection does not explain the gap. |
| Figure 7 | Across nine energies, mean relative delay differences are 13.65%, 25.89%, 34.39%, 30.60%, 28.63% for coherent, squeezed, and entangled $n = 2, 4, 8$. | My mean delays are higher for every family, although all 45 internal ARL checks pass. |
| Figure 11(b) | My single-photon conditional delay is $54.640 \pm 0.129$ pulses (one standard error). The released array has a signed mean of 48.034. | The array includes two negative values. I need the original event definitions to compare the means fairly. |

I checked the ARL for all 61 Figure 3/7 settings against the two-million-pulse target. The released files do not include the original thresholds, per-point standard errors or raw first-alarm records. Following the paper's caption, I treat $n = 5000$ as the observation horizon rather than the number of trials.

For Figure 5, I used the v2 settings $N = 100$, $N_a = 1$, $\eta_{\text{tap}} = 0.9$, fractional amplitude error 0.01 and phase variance 0.01 (SD 0.1 rad). The energy-matched coherent probe therefore has 101 photons. A ten-photon fit comes closer to the plot, but I have no evidence that the authors used that energy.

For Figure 11(b), my 30,000 trials give 17 early alarms and no post-change censoring, with $\nu = 1000$ and horizon 5000. The released vector contains $-844$ and $-129$. If these are complete

first-alarm latencies, removing negative entries gives a mean of 50.181 pulses. The remaining gap is 4.459 pulses, with bootstrap interval $[2.665, 6.206]$ under the independence and missing-event assumptions used in this analysis. This still does not explain the difference.

I also distinguish infinite ideal Kennedy KL from immediate detection: at finite energy, detection still waits for a changed pulse to produce a click.

## 5 Further checks

I also used the implementation to examine a few practical questions.

- **Rare false alarms.** Regenerative mixture proposals and independent holdout runs support 25 points on an exploratory grid, after checking effective sample size and weight concentration. I kept the earlier failed records.
- **Channel mismatch.** With the detector set to $\eta_1 = 0.9$, changing the true transmission from 0.899 to 0.901 changes classical ARL from about 1.31 to 3.01 million pulses. Even this small mismatch matters for false alarms.
- **Conservative thresholds.** On 25 sampled settings, adjusted approximate lower ARLs exceed two million. The added delay is 3.61–4.39 pulses for the classical detector and 1.12–2.33 for the squeezed/entangled examples. These checks cover the sampled settings, not every value between them.
- **Ideal dual-rail model.** I implemented the Appendix E.2 idea using Kraus loss maps, arbitrary qubit states and heralded survival. Uniform loss preserves the surviving state; unequal loss can distort it. The model accounts for two-bin timing, and two holdouts support the two-million-bin ARL target. It does not include cavity dynamics or a QKD security analysis.

I treat these as checks and extensions of this implementation. I have not established whether they are new research results.

## 6 Running the code and remaining questions

The recorded verification includes 428 tests, 1,869 core checks and 984 additional checks, plus file-hash checks for 739 raw arrays and 572 metadata files. These recalculate the saved results; they do not rerun every random trajectory. I separately reran the seven-cell introductory notebook. The dependency snapshot is in `requirements.txt`, and the original author files are in `data/authors`.

After installing the package, the following commands run the tests, execute the introductory notebook and check the saved evidence. Each output folder should be new:

```
python -m pytest -q
python -m lossqcd.lesson --output results/lesson_new
python -m lossqcd.evidence --scope all
python -m lossqcd.report --output results/review_new
python -m lossqcd.report --check results/review_new
```

The two evidence archives require approximately 1.8 GiB of downloads. The README and results report give installation steps and commands for generating new simulation ensembles.

The next step is to clarify the Figure 3/7/11 thresholds and event definitions, the Figure 4 fixed-seed calculation and the Figure 5 plotting calculation. I have not proved worst-case optimality for the correlated detector, built a capacity-achieving communication code or solved

the optimal tradeoff between communication and detection. Both the paper and this project are numerical and theoretical, so the remaining numerical gaps cannot be attributed to missing hardware.

# A Plots

These plots were generated by my implementation. The captions identify the paper figures being compared and note where agreement is incomplete.

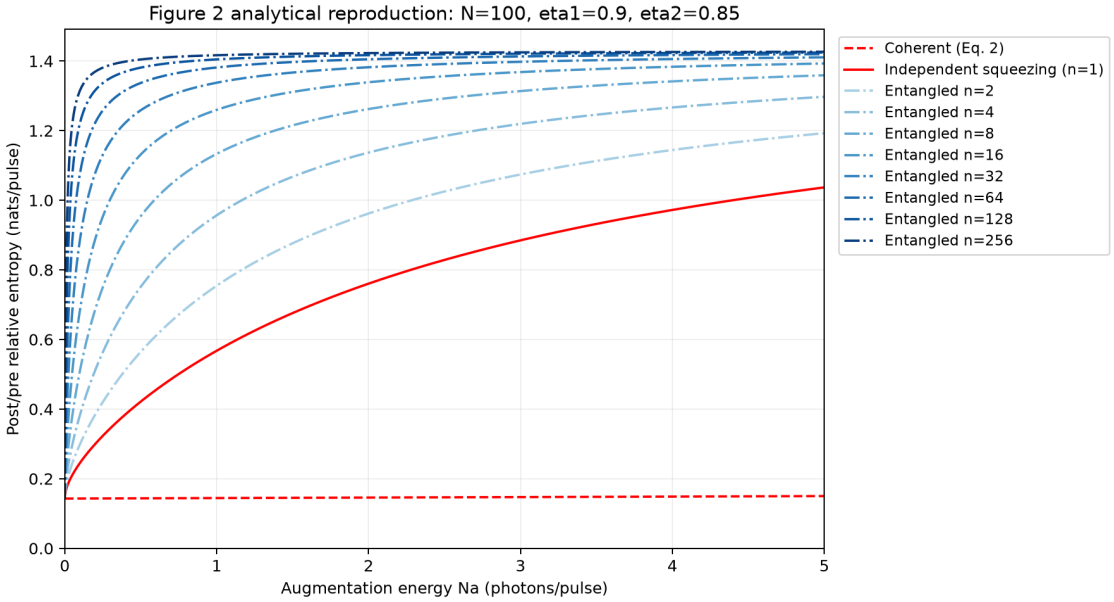



Figure 1: My Figure 2 calculation for coherent, squeezed and entangled probes. The formulas agree; the released dataset has no numeric array for a direct comparison.

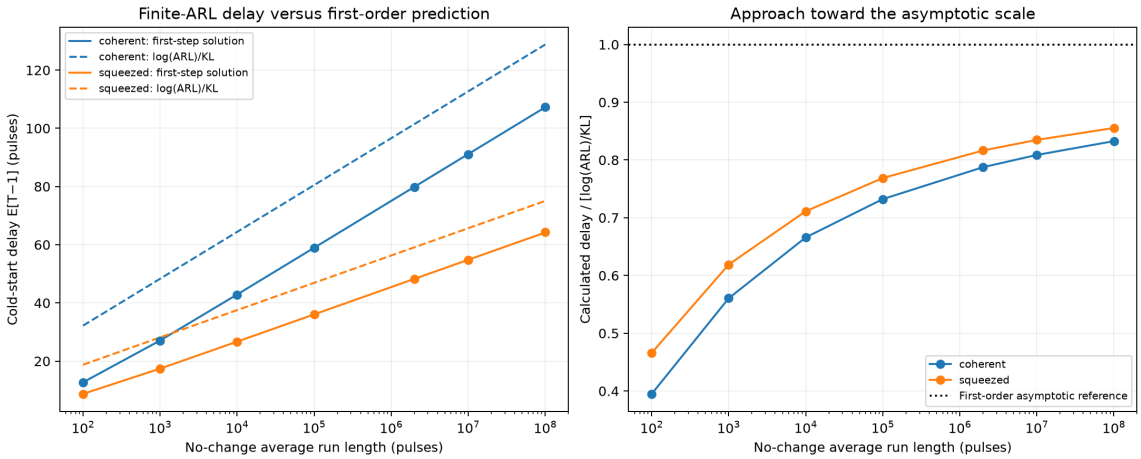



Figure 2: Delay scaling as the false-alarm ARL increases. The high-ARL slopes approach inverse-KL scaling, with finite-start offsets still visible.

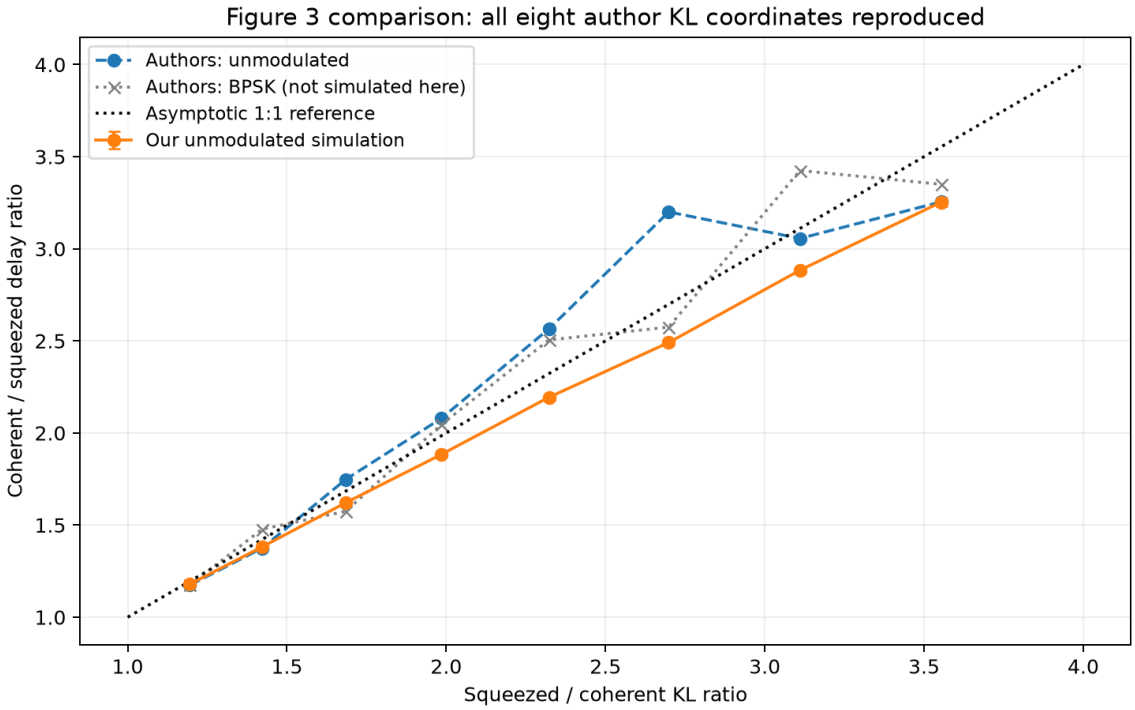



Figure 3: My Figure 3 delay-improvement ratios alongside the released values. The KL coordinates agree, but the delay ratios differ.

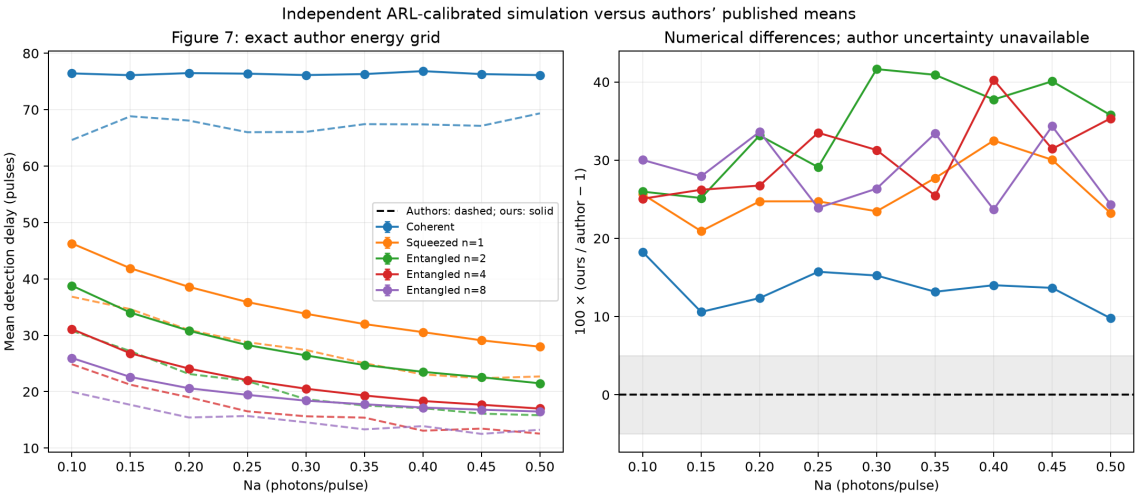



Figure 4: Figure 7 comparison after calibrating my detectors to the false-alarm target. My average delays are higher for each family.

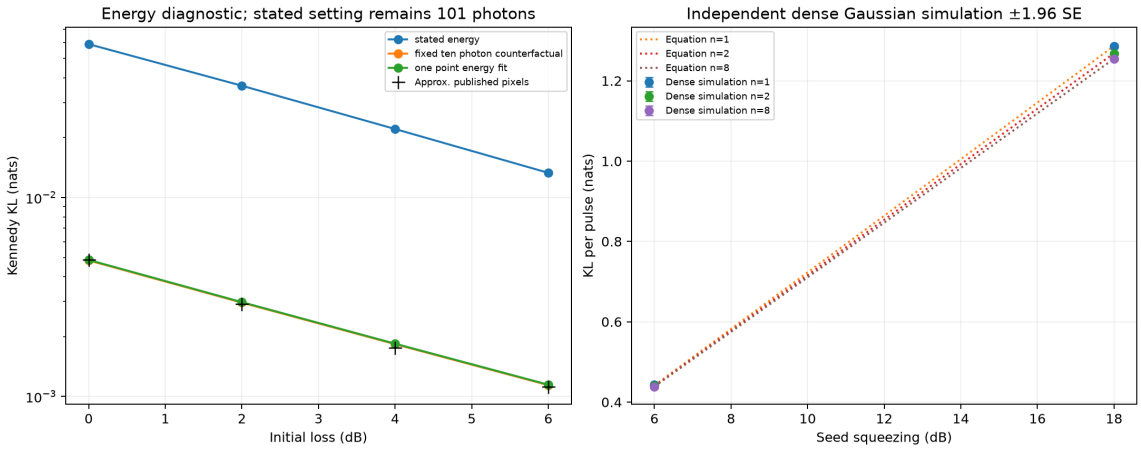



Figure 5: Checks of the Figure 4 fixed-seed and Figure 5 phase-noise calculations. The fitted energies are exploratory and are not verified author settings.

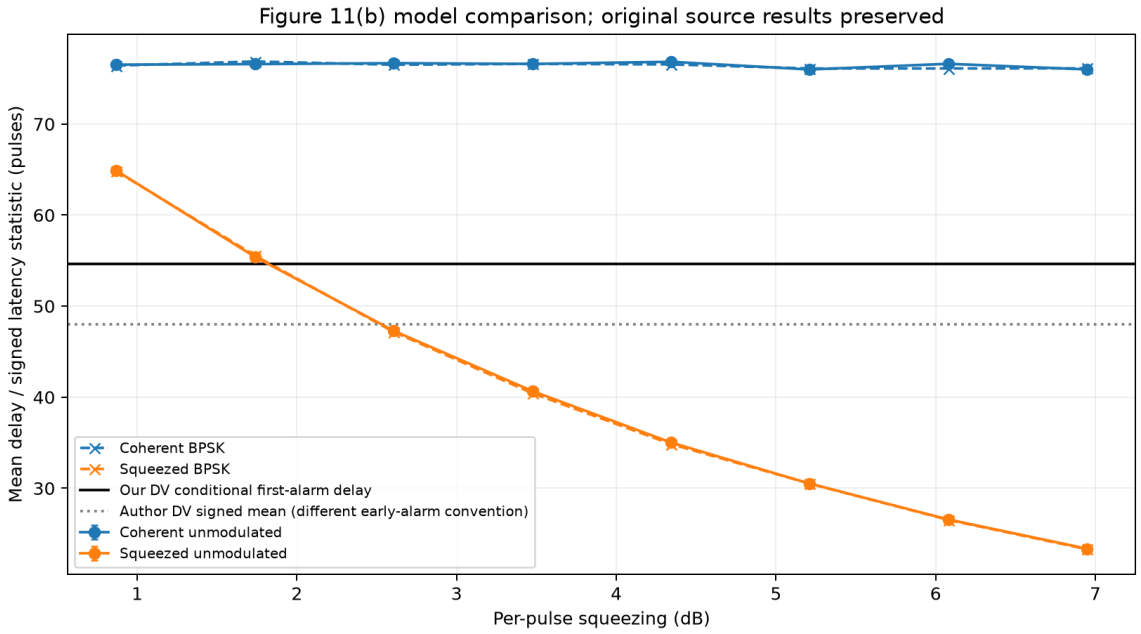



Figure 6: Single-photon and squeezed-light results for the Figure 11 comparison. The single-photon entropy agrees, while the delay comparison remains unresolved.

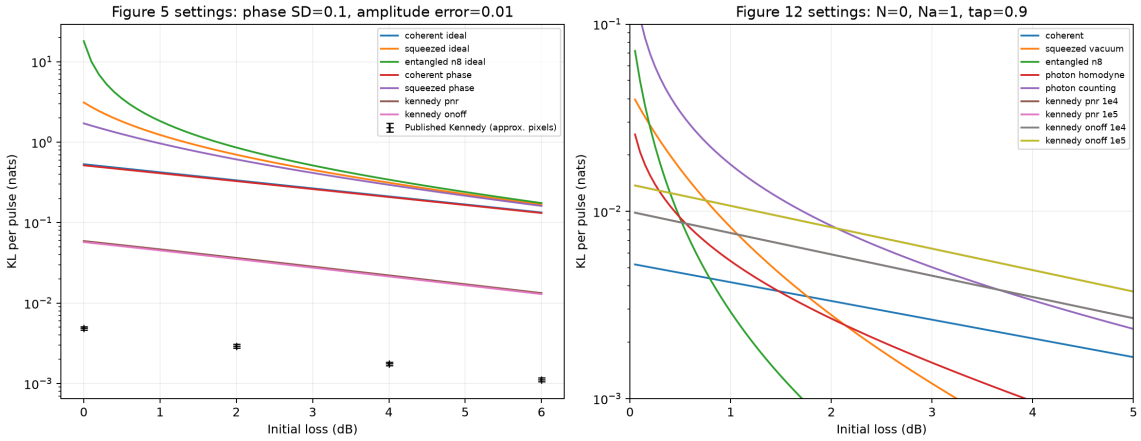



Figure 7: Weak-probe curves for the Figure 12 comparison. I checked seven approximate published values at 1 dB, rather than the full curves.

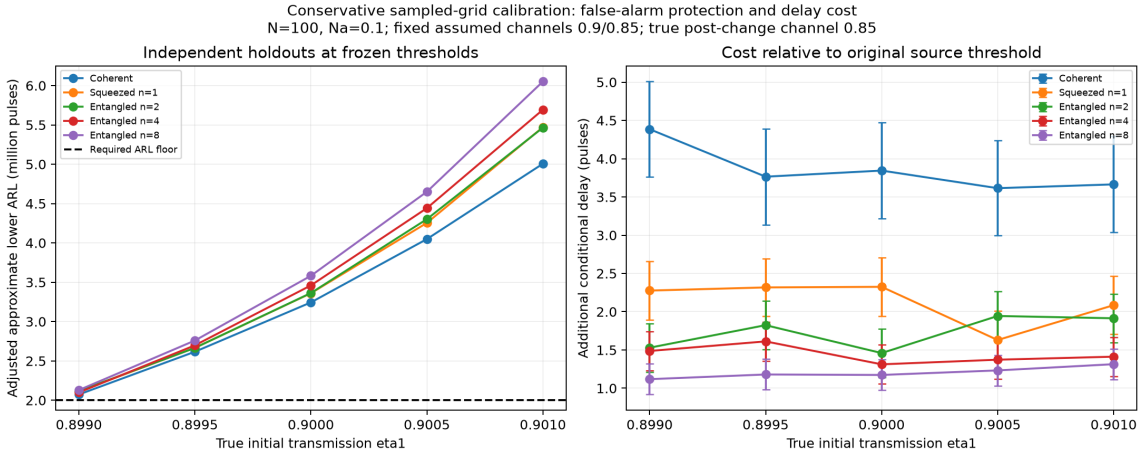



Figure 8: The delay cost of more conservative thresholds at the sampled initial-channel settings.

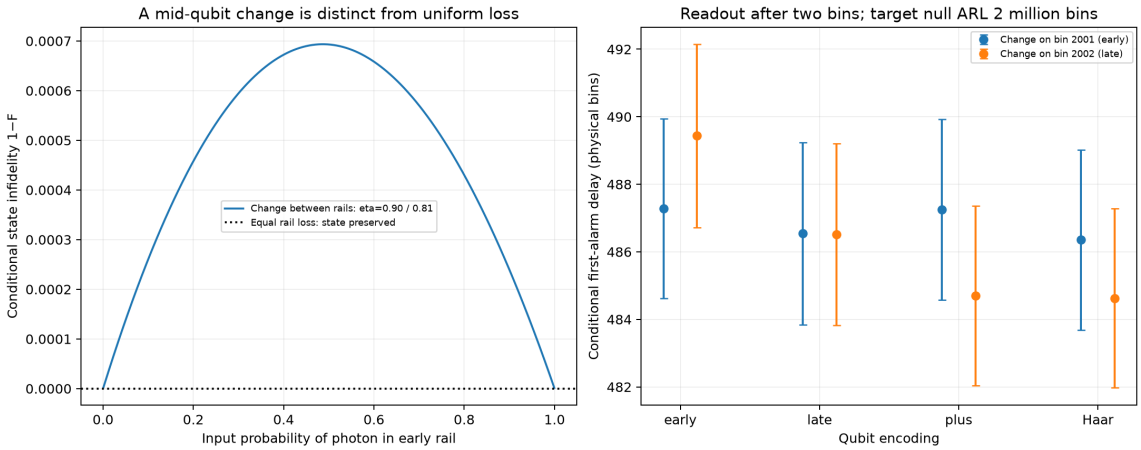



Figure 9: My ideal dual-rail simulation based on Appendix E.2. Cavity dynamics and QKD security are not part of this model.

# B Code examples

The excerpts below show the main detection steps from revision `d1e545e`. I shortened long docstrings for print without changing the calculations. The first four excerpts belong to their original modules; the last is a runnable example after package installation. The full squeezed and entangled models remain in the repository.

## B.1 Gaussian likelihood and KL

I use this class for the scalar Gaussian measurements. `post.llr(x, pre)` scores a measurement, and `post.kl(pre)` returns its expected post-change evidence in nats. The module imports `dataclass` and NumPy as `np`.
Source: `src/lossqcd/models.py`.

```
@dataclass(frozen=True)
class Gaussian:
    """Scalar homodyne law with finite mean and strictly positive variance."""

    mean: float
    variance: float

    def __post_init__(self):
        if not np.isfinite(self.mean):
            raise ValueError("mean must be finite")
        if not np.isfinite(self.variance) or self.variance <= 0:
            raise ValueError("variance must be finite and positive")

    def logpdf(self, x):
        """Evaluate the Gaussian log density elementwise, preserving the input shape."""
        x = np.asarray(x, dtype=float)
        return -0.5 * (np.log(2 * np.pi * self.variance) + (x - self.mean) ** 2 / self.
    variance)

    def llr(self, x, pre):
        """log p_self(x)/p_pre(x); self is the post-change distribution."""
        y = np.asarray(x, dtype=float) - pre.mean
        delta = self.mean - pre.mean
        return (
            0.5 * np.log(pre.variance / self.variance)
            + 0.5 * y**2 * (1 / pre.variance - 1 / self.variance)
            + y * delta / self.variance
            - delta**2 / (2 * self.variance)
        )

    def kl(self, pre):
        """D(self || pre), in nats per observation."""
        ratio = self.variance / pre.variance
        return float(0.5 * (ratio - 1 - np.log(ratio) + (self.mean - pre.mean) ** 2 / pre.
    variance))
```

## B.2 CUSUM stopping rule

This is the Page recursion used in the scalar detector. It returns the first strict threshold crossing, counted from one, or `None` if no alarm is seen.
Source: `src/lossqcd/cusum.py`.

```
def validate_threshold(threshold):
    """Require a finite, nonnegative Page threshold; alarms use a strict crossing."""
    if not np.isfinite(threshold) or threshold < 0:
        raise ValueError("threshold must be finite and nonnegative")


def page_cusum(llrs):
    """Return W_1,...,W_n. Empty suffix is allowed (the reflection at zero)."""
    llrs = np.asarray(llrs, dtype=float)
    if llrs.ndim != 1 or not np.all(np.isfinite(llrs)):
        raise ValueError("llrs must be a finite one-dimensional sequence")
    out = np.empty(llrs.size)
    w = 0.0
    for i, increment in enumerate(llrs):
        w = max(0.0, w + increment)
        out[i] = w
    return out


def stopping_time(llrs, threshold):
    """One-based first STRICT crossing; None if not observed in the sequence."""
    validate_threshold(threshold)
    crossings = np.flatnonzero(page_cusum(llrs) > threshold)
    return int(crossings[0] + 1) if crossings.size else None
```

## B.3 Simulating first alarms

Once a run alarms, I remove it from the active set. The event flag separates a real alarm on the last pulse from a run that reached the horizon without alarming. The function returns a `Runs` record.
Source: `src/lossqcd/simulation.py`.

```
def simulate_stopping_times(
    pre, post, threshold, trials=2000, horizon=100000,
    change_at=None, seed=20261020,
    sampling_pre=None, sampling_post=None,
):
    """Sample Gaussian observations and score with the assumed pre/post laws."""
    validate_threshold(threshold)
    for name, value in (("trials", trials), ("horizon", horizon)):
        if not isinstance(value, (int, np.integer)) or value < 1:
            raise ValueError(f"{name} must be a positive integer")
    if change_at is not None and (
        not isinstance(change_at, (int, np.integer)) or not 1 <= change_at <= horizon
    ):
        raise ValueError("change_at must be within the observation window")
    rng = np.random.default_rng(seed)
    actual_pre = pre if sampling_pre is None else sampling_pre
    actual_post = post if sampling_post is None else sampling_post
    stop = np.full(trials, horizon, dtype=np.int64)
    event = np.zeros(trials, dtype=bool)
    active = np.arange(trials)
    w = np.zeros(trials)
    for t in range(1, horizon + 1):
        dist = actual_post if change_at is not None and t >= change_at else actual_pre
        x = rng.normal(dist.mean, np.sqrt(dist.variance), active.size)
        w[active] = np.maximum(0.0, w[active] + post.llr(x, pre))
        crossed = w[active] > threshold
        stopped = active[crossed]
        stop[stopped], event[stopped] = t, True
        # Retire the first alarm permanently; later samples cannot replace it.
        active = active[~crossed]
        if not active.size:
            break
    return Runs(stop, event, horizon, change_at)
```

## B.4 Counting false alarms and delays

I count early alarms before calculating delay among the surviving runs. The helper `_mean_se` returns the sample mean and Monte Carlo standard error. If any survivor is censored, I report only the restricted delay.
Source: `src/lossqcd/simulation.py`.

```
def summarize_delay(runs):
    """E[T-nu | T>=nu], with zero delay for detection on observation nu."""
    nu = runs.change_at
    if nu is None:
        raise ValueError("delay requires a finite change point")
    false = runs.event & (runs.stop < nu)
    survivors = ~false
    censored = int(np.sum(survivors & ~runs.event))
    mean, se = _mean_se(runs.stop[survivors] - nu)
    probability = float(np.mean(false))
    return {
        "trials": len(runs.stop),
        "change_at": nu,
        "horizon": runs.horizon,
        "false_alarms": int(false.sum()),
        "false_alarm_probability": probability,
        "false_alarm_probability_se": float(np.sqrt(probability * (1 - probability) / len(
    false))),
        "survivors": int(survivors.sum()),
        "post_change_censored": censored,
        "conditional_delay": mean if censored == 0 else None,
        "conditional_delay_se": se if censored == 0 else None,
        "restricted_conditional_delay": mean,
        "restricted_conditional_delay_se": se,
    }
```

### B.5 Example run

Save this as `example.py` after installing the package with `python -m pip install -e '.[dev]'`. It calculates two KL values and runs separate no-change and changed simulations. The threshold is chosen for a small demonstration; it is not calibrated to the paper's false-alarm target. The printed early-alarm and censoring counts are needed to interpret the means.

```python
import json
import numpy as np

from lossqcd.models import coherent_pair, squeezed_pair
from lossqcd.simulation import (
    simulate_stopping_times, summarize_arl, summarize_delay,
)

# Formula comparison at one Figure 2 energy point.
for model in (coherent_pair, squeezed_pair):
    pre, post = model(N=100, Na=0.1, eta1=0.9, eta2=0.85)
    print(model.__name__, "KL:", post.kl(pre))

# Teaching example only: this threshold is not calibrated.
pre, post = coherent_pair(N=100, Na=0.1, eta1=0.9, eta2=0.85)
seeds = np.random.SeedSequence(20261029).spawn(2)
settings = dict(threshold=3.0, trials=1000, horizon=10000)
null = simulate_stopping_times(pre, post, seed=seeds[0], **settings)
changed = simulate_stopping_times(
    pre, post, seed=seeds[1], change_at=100, **settings,
)
print(json.dumps(summarize_arl(null), indent=2))
print(json.dumps(summarize_delay(changed), indent=2))
```

## References


[1] S. Guha, T. C. John, Z. Gong and P. Basu, Quantum-enhanced quickest change detection of transmission loss, arXiv:2503.12276v2 (2025). `https://arxiv.org/abs/2503.12276v2`.

[2] S. Guha and P. Basu, *Dataset accompanying PRL publication on quantum-enhanced quickest change detection of transmission loss*, Zenodo, v1 (2025). doi:10.5281/zenodo.17460928.

[3] E. S. Page, Continuous inspection schemes, *Biometrika* **41**(1–2), 100–115 (1954). doi:10.1093/biomet/41.1-2.100.

[4] G. Lorden, Procedures for reacting to a change in distribution, *The Annals of Mathematical Statistics* **42**(6), 1897–1908 (1971). doi:10.1214/aoms/1177693055.